

BeeQoS: A Cloud-Native QoS System for Adaptive and Scalable Multi-Priority Bandwidth Guarantees

Jinyao Liu
Shandong University
Qingdao, China
jacelau@mail.sdu.edu.cn

Chaoqun Li
Shandong University
Qingdao, China
chaoqunli@mail.sdu.edu.cn

Feng Li
Shandong University
Qingdao, China
Inspur Cloud Information Technology
Co., Ltd
Jinan, China
fli@sdu.edu.cn

Si Wu
Shandong University
Qingdao, China
kobesi@sdu.edu.cn

Hongjing Yu
Shandong University
Qingdao, China
yuhj@mail.sdu.edu.cn

Pengfei Hu*
Shandong University
Qingdao, China
Quancheng Laboratory
Jinan, China
phu@sdu.edu.cn

Haoyuan Ma
Shandong University
Qingdao, China
haoyuanma@mail.sdu.edu.cn

Dingyi Jia
Shandong University
Qingdao, China
dingyijia@mail.sdu.edu.cn

Abstract

Modern cloud applications, from interactive web services to mobile and WoT workloads, generate highly dynamic multi-tenant network demands. Guaranteeing priority-aware bandwidth remains challenging: legacy shapers like Linux Traffic Control Hierarchical Token Bucket are static and unscalable, while cloud-native solutions such as Cilium offer only coarse-grained rate limiting. We present BeeQoS, a cloud-native QoS system that delivers low-latency, adaptive, and scalable multi-priority bandwidth guarantees. BeeQoS consists of an eBPF-powered data plane for high-performance, fine-grained per-packet shaping, a demand-aware control plane that senses real-time flow requirements and adaptively reallocates bandwidth, and seamless Kubernetes integration for expressive policy specification and cluster-wide scalable deployment. Evaluation shows that BeeQoS scales to 1K+ flows with stable performance, boosts high-/medium-priority throughput by 14.6%/36.4%, cuts median latency by 72.4%, reduces deployment overhead, and improves video QoE by 27.3% over state-of-practice baselines.

CCS Concepts

• **Networks** → *Network management*; **Cloud computing**.

Keywords

QoS, eBPF, Cloud-Native, Traffic Shaping, Datacenter Networking

*Corresponding author: Pengfei Hu.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792487>

ACM Reference Format:

Jinyao Liu, Si Wu, Haoyuan Ma, Chaoqun Li, Hongjing Yu, Dingyi Jia, Feng Li, and Pengfei Hu. 2026. BeeQoS: A Cloud-Native QoS System for Adaptive and Scalable Multi-Priority Bandwidth Guarantees. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3774904.3792487>

Resource Availability:

The source code of this paper has been made publicly available at <https://github.com/IIC-SIG-MLsys/BeeQoS>.

1 Introduction

The modern web ecosystem is increasingly hosted on cloud infrastructures, where interactive Web, Mobile, and Web of Things (WoT) workloads colocate in multi-tenant environments [13, 23, 38]. These applications exhibit highly diverse traffic patterns and heterogeneous QoS requirements: latency-sensitive front-end services generate bursty traffic and demand responsiveness, while background analytics or batch jobs transfer bulk traffic and primarily require minimum throughput guarantees. Such heterogeneity, i.e., bursty web requests coexisting with bulk data analytics, creates highly variable and often unpredictable demand, a phenomenon also observed in domains such as in-network anomaly detection and event-aware traffic prediction [36, 37]. Consequently, workloads must be treated with differentiated priorities, ensuring bandwidth guarantees for high-priority, web-facing services while allowing surplus capacity to be opportunistically reclaimed by lower-priority services to maximize efficiency and fairness [27, 39]. Fine-grained and adaptive QoS enforcement is therefore a critical foundation for reliable and efficient cloud services.

To enforce QoS, traditional traffic shaping mechanisms such as Hierarchical Token Bucket (HTB) remain widely deployed. HTB, implemented in Linux Traffic Control (TC), organizes traffic classes

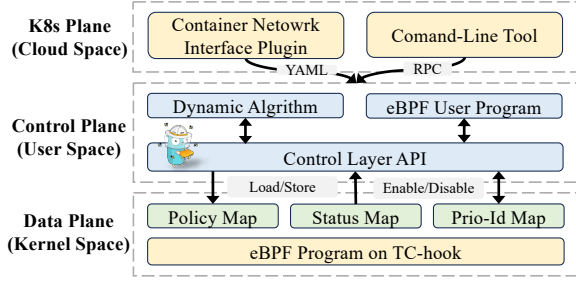


Figure 1: Overview of the BeeQoS.

into a hierarchy where each class specifies both a guaranteed rate and a maximum ceiling [8, 15]. Each child class first receives its guaranteed rate; if the parent has excess capacity, eligible children may borrow unused tokens up to their ceiling. While effective in small-scale environments, HTB exhibits fundamental limitations in cloud-native settings: First (**non-scalable**), it relies on manually configured hierarchies, which do not scale to thousands of services with highly dynamic lifecycles. Second (**overhead**), its queuing and classification logic introduces substantial latency overhead that grows with hierarchy depth [20]. Finally (**non-adaptive**), its static configuration model is fundamentally misaligned with elastic cloud deployments where workloads scale up or down within seconds.

Meanwhile, Kubernetes, the de facto orchestration framework for cloud-native clusters, provides only limited support for network QoS. Its official bandwidth plugin exposes an experimental `BandwidthRateLimiting` feature via annotations, but this mechanism is TC-based, coarse-grained, and incurs high overhead, often degrading performance [5, 10]. To overcome these limitations, recent efforts have turned to kernel programmability, particularly with *extended Berkeley Packet Filter (eBPF)* [29]. eBPF enables developers to safely execute custom programs directly in the Linux kernel without modifying kernel code, providing high-performance, fine-grained per-packet processing while ensuring strong memory safety. Systems such as Cilium [5] integrate TC and eBPF-based mechanisms to implement egress rate limiting. However, these solutions remain constrained. First, **they are neither adaptive nor priority-aware**. Their configurations are typically static and applied per pod or container, which prevents dynamic reallocation in response to workload changes and fails to enforce differentiated priorities across workloads. Second, **they lack mechanisms for bandwidth guarantees**. They cannot express richer policies, such as hierarchical prioritization or per-flow guarantees, both of which are essential for production-grade bandwidth assurance in cloud environments.

To address these limitations, we urgently require a programmable traffic shaping scheme that delivers high-performance, priority-aware bandwidth guarantees, adapts to dynamic cloud environments. Achieving this vision, however, raises three fundamental challenges: (1) *High-performance data path*: Implementing fine-grained, per-flow traffic control with minimal overhead. (2) *Dynamic, priority-aware bandwidth controlling*: Enforcing adaptive, demand-driven policies within the constrained eBPF environment. (3) *Efficient control/data plane coordination*: Maintaining lightweight communication to propagate policy updates and synchronize flow

state. Collectively, these challenges shows the gap between existing static rate limiters and the need for a programmable, demand-aware traffic shaping framework suitable for cloud-native environments.

We present *BeeQoS*, a high-performance, adaptive and scalable QoS system that provides multi-priority bandwidth guarantees in dynamic cluster environments. As illustrated in Figure 1, BeeQoS consists of three key layers: (1) an eBPF-powered data plane that performs per-packet shaping and flow sampling, (2) a control plane daemon that dynamically allocates bandwidth based on real-time application demand sensing, and (3) a Kubernetes integration layer that enables policy specification via annotations and seamless cluster-wide deployment. By unifying these components, BeeQoS achieves fine-grained yet high-performance traffic shaping that adapts to instantaneous network conditions. This design not only enforces robust priority-aware bandwidth guarantees but also maximizes utilization by reclaiming unused capacity. It achieves all these benefits without modifying the kernel code.

We conduct a comprehensive benchmarking study of BeeQoS using real-world cloud workloads. Our evaluation demonstrates that BeeQoS significantly outperforms HTB- [8] and Cilium-based [5] baselines in terms of bandwidth utilization, service latency and fairness. In summary, this paper makes the following contributions:

- **BeeQoS**. We design and implement BeeQoS, a cloud-native QoS system that unifies an eBPF-powered data plane, a demand-aware control plane, and Kubernetes integration to deliver high-performance, adaptive, scalable, and priority-aware bandwidth guarantees.
- **Dynamic bandwidth reallocation**. We propose an algorithm that senses real-time flow demand and adaptively reallocates bandwidth, enabling elastic yet predictable multi-priority guarantees beyond the static enforcement model of existing approaches.
- **Extensive evaluation**. We perform comprehensive experiments across diverse workloads and deployment settings, showing that BeeQoS enforces accurate multi-priority bandwidth guarantees and outperforms state-of-practice baselines: improving high-/medium-priority throughput by up to 14.6%/36.4%, reducing median latency by 72.4%, sustaining near-saturation throughput with stable fairness at 1K+ flows, cutting deployment overhead by an order of magnitude, and boosting video QoE by up to 27.3%.

2 Background and Motivation

2.1 Programmable Kernel with eBPF

eBPF has emerged as an in-kernel runtime that brings flexible packet processing to Linux while enforcing strong safety guarantees. Programs are written in a restricted C-like language, compiled via LLVM into bytecode, and verified before being just-in-time compiled into native instructions. By attaching to hooks such as XDP and TC [12, 17, 18, 24], eBPF enables lightweight traffic manipulation and fine-grained observability, leveraged by systems like Cilium and Katran [5, 9, 30, 32].

Figure 2 illustrates the eBPF workflow. Kernel programs are loaded via the `bpfl()` syscall and verified before JIT compilation. User-space programs manage these kernel programs and interact with them through maps [28], enabling stateful logic, per-packet processing, and controlled interaction with kernel functions. This

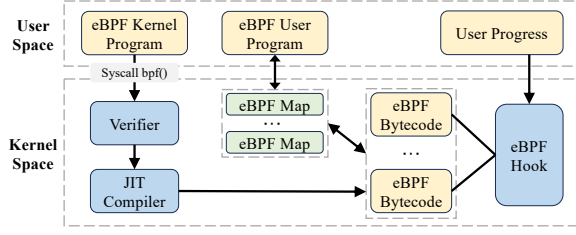


Figure 2: eBPF Workflow.

separation of responsibilities allows eBPF to provide safe, low-overhead, and high-performance programmability directly within the kernel.

2.2 Traditional QoS Mechanisms

Linux’s TC subsystem implements packet scheduling via qdiscs, classifiers, and filters, with TBF and HTB being the most common qdiscs [8]. TBF is simple and easy to deploy (hence common in container plugins [6]), but it treats all traffic as a single aggregate, is strictly non-work-conserving, and offers no per-flow prioritization. As a result, aggressive flows can monopolize capacity, while idle periods lead to underutilization [25]. HTB adds hierarchical, class-based guarantees and opportunistic borrowing among siblings, making it more suitable for structured QoS. However, it requires explicit, manually maintained hierarchies, incurs per-packet overhead that grows with depth, and limits sharing and fairness across the entire hierarchy [8, 20, 26]. This need for complex manual configuration, combined with static structure, processing overhead, and restricted sharing, makes TC poorly suited to highly dynamic, large-scale cloud environments.

Kubernetes itself provides little native network QoS; bandwidth control is typically delegated to Container Network Interface (CNI) plugins, which are responsible for configuring networking for Pods and implementing features such as IP assignment, routing, and traffic shaping. Most notably, the experimental TBF-based bandwidth plugin enforces per-Pod caps [6, 10]. This approach yields coarse, static limits and inherits TBF’s lack of prioritization. Third-party CNIs (e.g., Calico, Flannel, vendor CNIs) usually wrap TC/iptables workflows or lose functionality under eBPF dataplanes [3–5, 7, 13, 19]. For instance, Calico annotations are not supported when running with its eBPF dataplane, while Flannel falls back to static TBF caps. Cilium introduces a more advanced eBPF-based per-Pod shaping mechanism using an Earliest Departure Time (EDT) model, which reduces queuing delay on multi-core NICs [5, 14], but its configuration remains largely static and per-Pod, lacking hierarchical policies or dynamic multi-priority reallocation. Overall, today’s Kubernetes-native mechanisms remain either coarse-grained or rigidly static, and thus fail to meet the demands of large-scale and dynamic cloud environments.

2.3 Motivation and Challenges

To date, no existing solution simultaneously provides fine-grained multi-priority guarantees, dynamic redistribution of unused capacity, and scalable operation across thousands of ephemeral workloads. This gap is particularly pressing in cloud-native clusters

where services with different priorities coexist on shared infrastructure, directly motivating our design.

We argue that a new programmable traffic shaping framework is needed: one that achieves high-performance, priority-aware bandwidth guarantees while adapting to dynamic cloud environments and scaling to thousands of services. Achieving this vision raises the following three key challenges:

- (1) *High-performance data path design.* Fine-grained traffic shaping must be achieved without compromising throughput. This requires eBPF programs that can parse traffic flows, monitor per-flow statistics, and enforce packet-level rate control, all with minimal overhead and latency in the fast path.
- (2) *Dynamic and priority-aware bandwidth control.* The eBPF data plane cannot support complex schedulers (e.g., HTB), while conventional rate limiters only cap flow rates without reclaiming unused bandwidth. As a result, achieving demand- and priority-aware control in the data plane is infeasible, motivating a control plane for real-time demand sensing and adaptive redistribution.
- (3) *Efficient control/data plane coordination.* Lastly, a lightweight communication channel between control and data planes to propagate frequent policy updates and synchronize flow state is required. This channel must achieve high update rates with minimal contention and overhead, avoiding new bottlenecks.

3 BeeQoS Design

The design of BeeQoS is guided by the principle of providing fine-grained, priority-aware, adaptive, and scalable bandwidth guarantees in dynamic, multi-tenant cloud-native environments. BeeQoS achieves this by vertically integrating three distinct layers: a high-performance eBPF-based data plane operating in kernel space, an intelligent user-space control plane, and a cloud-native integration layer for seamless deployment and policy management within Kubernetes. The core of BeeQoS, shown in Figure 3, is a closed-loop feedback mechanism. The data plane, implemented as an eBPF program attached to the TC egress hook, intercepts every outgoing packet. It performs lightweight flow monitoring and enforces per-packet shaping decisions based on policies stored in eBPF maps. Concurrently, the control plane, running as a DaemonSet on each Kubernetes node, aggregates real-time flow statistics sampled from the data plane. It continuously senses application bandwidth demands and, using a priority-aware algorithm, computes precise rate allocations. These allocations are then propagated back to the data plane by updating the corresponding eBPF map entries. Finally, the Kubernetes integration layer provides a declarative API, allowing users to specify QoS requirements via simple pod annotations. These high-level intents are automatically translated into low-level policies enforced by the control and data planes.

3.1 Data Path: High-Performance Packet Pacing

We designed a high-performance kernel data path using eBPF that enforces per-flow QoS at line rate with minimal overhead. Our data path is built entirely on eBPF programs attached to the Linux TC subsystem on the host NIC. It combines three key techniques: high-granularity flow identification and sampling suitable for cloud-native workloads, precise packet pacing and a unified shaping that handles both ingress and egress traffic symmetrically.

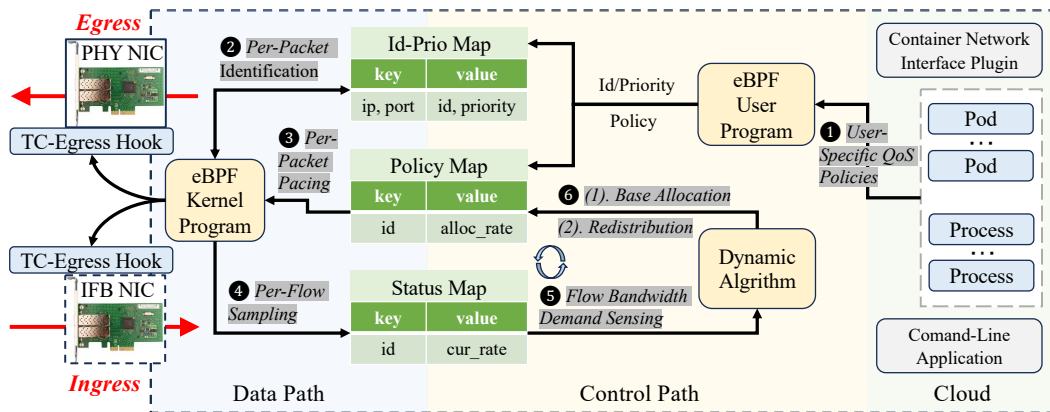


Figure 3: BeeQoS Architecture with eBPF Data Plane, Intelligent Control Plane, and Kubernetes Integration.

High-Granularity Flow Identification and Sampling. (**Identification**) Since the eBPF program executes in the kernel and processes every packet, the data path must first determine each packet’s flow and its associated policies. BeeQoS achieves this through a hierarchical lookup in an id-prio eBPF map that stores per-flow identifiers, priorities, and rate-limiting rules configured by the control plane. A fast IP-based match associates packets with container pods; if this fails, a finer-grained lookup on (IP, port) distinguishes individual processes, depending on whether the flow acts as client or server. This enables BeeQoS to support both pod-level and process-level flow identification granularity.

(Sampling) Once a packet is identified, per-flow counters are updated via atomic operations, enabling continuous and lightweight traffic measurement. To provide stable inputs for higher-level scheduling, the data plane periodically computes the instantaneous rate x_t over a configurable *sampling interval* (e.g., 10ms) and applies an Exponentially Weighted Moving Average (EMA), $s_t = \alpha x_t + (1 - \alpha)s_{t-1}$, to smooth transient bursts. The resulting values are stored in a status eBPF map and consumed by the control plane, ensuring that allocation and prioritization decisions rely on robust, pre-processed metrics with negligible per-packet cost.

eBPF-based EDT Pacing. Traditional queue-based packet shaping maintains queues and relies on complex in-kernel schedulers, which cannot be implemented within the restricted eBPF runtime. In contrast, Linux has supported the Earliest Departure Time (EDT) model [14, 16]. Under EDT, the pacing task is reduced to assigning each packet a departure timestamp (`skb->tstamp`), while the kernel’s highly-optimized fq qdiscs enforce transmission according to these timestamps with nanosecond precision. BeeQoS leverages this mechanism to implement efficient, dynamic pacing in eBPF.

(Pacing) Our eBPF program cooperates with the kernel qdisc by performing a lightweight four-step procedure for each packet: (1) look up the flow’s allocate rate, dynamically configured by the control plane, from the policy eBPF map; (2) compute the pacing delay as $\text{delay} = \text{skb} \rightarrow \text{len} / \text{rate}$; (3) calculate the candidate departure time as $\text{EDT} = \max(\text{now}, \text{last_departure_time}) + \text{delay}$; and (4) stamp this value into $\text{skb} \rightarrow \text{tstamp}$. The packet is then passed to the fq qdisc, which uses the timestamps to enforce pacing. This separation of roles is key: the eBPF program performs only fast,

stateless calculations, while the kernel maintains precise timing. As a result, BeeQoS achieves accurate, low-overhead pacing with full support for dynamic control plane rate updates.

Ingress and Egress Unifying. The Linux kernel’s ingress TC hook is functionally limited: it cannot buffer or pace incoming packets. To provide symmetric QoS guarantees for both inbound and outbound traffic, BeeQoS implements a unified shaping by leveraging an Intermediate Functional Block (IFB) [12] virtual device.

(Unifying) For ingress traffic, a lightweight TC eBPF program attached to the physical interface’s ingress hook redirects packets to a dedicated IFB device using a mirrored egress redirect action. This redirection makes the ingress packets appear on the egress path of the IFB device, allowing the same EDT-based eBPF pacing program used on physical egress to be applied transparently. Consequently, both ingress and egress traffic are subjected to identical, fully-featured shaping logic. This design not only provides symmetric and stateful QoS capabilities but also creating a single, programmable, and direction-agnostic data path.

3.2 Control Path: Adaptive, Priority-Aware Bandwidth Allocation

Existing bandwidth control in Kubernetes is largely static: users specify guaranteed rates, but actual traffic often falls short, leaving capacity unclaimed. Traditional mechanisms either enforce rigid allocations (HTB) or cap peak rates without reclaiming slack (Cilium), resulting in wasted resources and poor utilization. To address this gap, BeeQoS introduces a control plane that complements the constrained eBPF datapath. While eBPF limiters can enforce simple per-flow caps, they cannot implement complex scheduling or dynamic reallocation. The control plane bridges this divide by interpreting user QoS policies and orchestrating adaptive bandwidth redistribution that the data path alone cannot achieve. At its core, the control plane operates as a periodic feedback loop: it samples per-flow real-time rate, compares observed utilization against guarantees, and reallocates any surplus according to flow priorities. This design ensures configured guarantees while reclaiming idle bandwidth to maximize utilization. In the following, we describe how the control plane supports (i) manual QoS policy configuration and (ii) dynamic, adaptive bandwidth allocation.

User-Specific QoS Policies. BeeQoS provides both a CNI plugin and a command-line tool for users to configure QoS policies. Users can express requirements via Pod annotations or CLI commands. When a Pod is created, the CNI plugin parses these specifications and translates them into entries in the eBPF policy maps. Each flow is associated with a priority and requested guarantee. By storing policies in eBPF maps, the control plane enables the kernel datapath to enforce fine-grained traffic shaping directly at line rate.

Dynamic and Priority-Aware Bandwidth Guarantees. For a flow p_i managed by the control path, the user specifies a guaranteed bandwidth req_rate_i , meaning the flow should receive at least req_rate_i if the flow needs it. However, the flow's actual demand cur_rate_i is not always equal to req_rate_i , but instead fluctuates with workload intensity. For example, a user may configure $req_rate_i = 200$ Mbps, while the flow's current demand is only 100 Mbps; allocating 200 Mbps in this case would waste 100 Mbps of capacity. BeeQoS addresses this by periodically assigning each flow an allocation rate $alloc_rate_i$, which the data path enforces as a rate limit. In the next control interval, the control path samples the instantaneous rate cur_rate_i from the data path, which evolves toward the allocated rate. Thus, a flow's true demand is not simply its configured guarantee, and its observed rate is directly shaped by the allocated bandwidth. When every flow is allocated bandwidth close to its actual demand, all flows are satisfied. After meeting the minimum demand of each flow, the NIC may still have residual capacity; the remaining bandwidth can be redistributed to flows with additional demand based on priorities, thereby improving overall utilization.

(Modeling) We model this as a *constrained, weighted resource allocation* problem, where the total bandwidth R of a Kubernetes node must be dynamically shared across flows $\mathcal{P}$. The goal is to maximize bandwidth utilization while ensuring each flow receives at least its true required bandwidth:

$$\text{Maximize } \sum_{i \in \mathcal{P}} cur_rate_i$$

However, the true demand of each flow's is unknown and time-varying, making static allocation models intractable. Inspired by dynamic exploration in congestion control algorithms such as BBR [21] and PCC [22], we instead infer demand through online probing. Accordingly, we decompose the dynamic, priority-aware bandwidth allocation problem into three stages (Algorithm 1):

- (1) **Demand sensing:** probe flows periodically to infer their true demand (lines 2-10).
- (2) **Base allocation:** assign minimum base bandwidth to satisfy each flow's true demand (lines 12-23).
- (3) **Redistribution:** allocate remaining bandwidth according to flow priorities (lines 25-34).

BeeQoS executes this process at each *control interval* as a looped algorithm, iteratively adjusting per-flow allocations. Over time, the system converges toward stable allocations. By reclaiming unused bandwidth from underutilized flows and redistributing it to flows with unmet demand, the controller maximizes utilization while preserving per-flow guarantees.

(Flow bandwidth demand sensing) To estimate each flow's demand state, the controller applies a throttle threshold θ at every *control interval* (i.e. $10 \times \text{control interval}$) to decide whether

Algorithm 1 Adaptive Bandwidth Allocation

Require: Flow set P , global minimum min_rate , throttle ratio θ , action factor β , selection factor γ , total bandwidth R ; For each flow p_i : req_rate_i , cur_rate_i , $alloc_rate_{i-1}$, priority w_i

Ensure: new $alloc_rate_i$ for all flows

```

1: // Stage 1: Demand sensing
2: for each flow  $p_i \in P$  do
3:   if  $cur\_rate_i < \theta \cdot alloc\_rate_{i-1}$  then
4:     mark  $p_i$  as Down
5:   else if  $alloc\_rate_{i-1} < req\_rate_i$  then
6:     mark  $p_i$  as Up-Below
7:   else
8:     mark  $p_i$  as Up-Above
9:   end if
10: end for
11: // Stage 2: Base allocation
12: for each flow  $p_i \in P$  do
13:   if state( $p_i$ ) is Down then
14:      $alloc\_rate_i = \max((1 - \beta) \cdot alloc\_rate_{i-1}, min\_rate)$ 
15:   else
16:      $alloc\_rate_i = \min((1 + \beta) \cdot alloc\_rate_{i-1}, req\_rate_i)$ 
17:     if  $\gamma \cdot req\_rate_i < alloc\_rate_i < req\_rate_i$  then
18:        $alloc\_rate_i = req\_rate_i$ 
19:       mark  $p_i$  as Up-Above
20:     end if
21:   end if
22:    $R \leftarrow R - alloc\_rate_i$ 
23: end for
24: // Stage 3: Redistribution
25: if  $R > 0$  then
26:   Compute total priority weight of flows marked Up-Above
27:   for each flow  $p_i$  marked Up-Above do
28:      $\Delta \leftarrow R \cdot \frac{w_i}{\text{total priority weight}} \cdot \frac{1}{\text{count of same-priority flows}}$ 
29:      $alloc\_rate_i \leftarrow alloc\_rate_i + \Delta$ 
30:      $R \leftarrow R - \Delta$ 
31:     if  $R \leq 0$  then break
32:   end if
33: end for
34: end if
35: Apply updated  $alloc\_rate_i$  to eBPF maps
36: Wait for next control interval and repeat

```

allocations should be increased or decreased. During each control interval, every flow is classified into one of three demand states by comparing its observed rate cur_rate_i with its previous allocation $alloc_rate_{i-1}$ and its user-specified guarantee req_rate_i :

- **Down:** $cur_rate_i < \theta \cdot alloc_rate_{i-1}$. The flow consumes significantly less bandwidth than allocated, indicating that part of its allocation can be reclaimed (lines 3-4).
- **Up-Below:** $cur_rate_i \geq \theta \cdot alloc_rate_{i-1}$ and $alloc_rate_{i-1} < req_rate_i$. The flow fully utilizes its current allocation but has not yet reached its guaranteed rate, making it a candidate for incremental increases (lines 5-6).
- **Up-Above:** $cur_rate_i \geq \theta \cdot alloc_rate_{i-1}$ and $alloc_rate_{i-1} \geq req_rate_i$. The flow has already satisfied its minimum guarantee

and continues to saturate its allocation, making it eligible for opportunistic surplus bandwidth (line 8)

The controller periodically derives these states by sampling each flow's cur_rate_i from the eBPF status map, thereby enabling informed allocation adjustments in subsequent intervals.

Adaptive two-stage bandwidth allocation: Following demand sensing, the allocation process proceeds in two stages. (**Base allocation**) To balance exploration for additional bandwidth with meeting observed demand, the first stage performs a *minimum base bandwidth allocation*. The controller updates each flow's $alloc_rate_i$ by multiplicatively adjusting its previous allocation $alloc_rate_{i-1}$ with factor β (i.e., $\pm\beta \cdot alloc_rate_{i-1}$), according to the flow's current state. To prevent starvation and prioritize guarantees, the updated rate is bounded within a predefined range: the lower bound is a system-wide minimum rate min_rate , while the upper bound is the user-specified guarantee req_rate_i (lines 13-16). This ensures that each flow's minimum demand is met without incurring wasteful allocations. Flows whose current allocation exceeds a fraction γ of their requested rate ($alloc_rate_i > \gamma \cdot req_rate_i$) are considered more urgent candidates for bandwidth guarantees, as their demand is approaching the requested rate. The controller immediately boosts their allocation to the requested rate req_rate_i and marks them as *Up-Above* (lines 17-20). By enforcing these bounds, the state classification from sensing stage is translated into concrete allocation decisions, which in turn drive the redistribution logic in second stage. As demand fluctuates, base allocation continually adapts, progressively steering each flow toward its true demand.

(**Priority bandwidth redistribution**) After the minimum base allocation, any remaining bandwidth is redistributed in a work-conserving, priority-aware manner. Only flows in an *Up-Above* state with sufficiently high demand are eligible for redistribution. Surplus bandwidth is first apportioned among priority tiers according to the user-configured weights (line 26), and then each tier's share is evenly divided among its constituent candidate flows (lines 27-33). This approach ensures that higher-priority flows are favored while maintaining fairness among flows within the same priority level, allowing the system to opportunistically utilize remaining bandwidth without violating guaranteed allocations.

3.3 Efficient Data/Control Plane Interaction

BeeQoS achieves high-performance data/control plane interaction by exposing all communication through eBPF maps. These maps are organized as *one-way pipes*: single-writer / single-reader maps that eliminate fast-path synchronization while ensuring the control plane receives timely and consistent information. We implement three logical eBPF maps with distinct roles and concrete types:

- **id-prio map (control→data):** a flow-identification table. Keys are IP/port pairs; values are 32-bit entries (high 16 bits = flow id, low 16 bits = priority). Implemented as `BPF_MAP_TYPE_LRU_HASH`, it is written infrequently by the control plane and read frequently by the data plane.
- **policy map (control→data):** stores per-flow bandwidth policies as a fixed array (`BPF_MAP_TYPE_ARRAY`) indexed by flow ID for fast data plane lookups. Like the id-prio map, it is updated at low frequency by the control plane and accessed at high frequency by the data plane.

- **status map (data→control, one-way pipe):** carries high frequency per-flow rate measurements from the data plane to the control plane. The data plane computes each flow's instantaneous rate at *sampling interval* which is then written into the map. The control plane reads these entries at a lower, tunable frequency, allowing it to consume the EMA-smoothed rates for stable reallocation decisions, while reflecting that policy changes require some time to influence the flow's observed rate.

4 Evaluation

In this section, We conduct a comprehensive benchmarking study of BeeQoS. Our evaluation aims to answer the following key questions:

- **Q1. Priority guarantees:** Does BeeQoS provide accurate multi-priority bandwidth assurance?
- **Q2. Adaptivity:** Can BeeQoS dynamically reallocate bandwidth under demand shifts?
- **Q3. Concurrency:** How does BeeQoS perform under increasing numbers of concurrent flows?
- **Q4. End-to-End Performance:** Does BeeQoS improve end-to-end application quality in mixed workloads?
- **Q5. Deployability:** Is BeeQoS simpler to configure and integrate across clusters compared to legacy QoS schemes?

The detail and analysis for **Q5** can be found in Appendix C.

4.1 Settings

Implementation. We implement BeeQoS in Go, with eBPF integration via libbpf v1.3 [11]. BeeQoS is packaged as a CNI plugin, built into a Docker image, and deployed through standard Kubernetes YAML manifests. During deployment, it is registered as an additional CNI plugin alongside Calico, enabling seamless integration with existing cluster networking. The core codebase consists of 4,720 lines of Go and 406 lines of eBPF C.

Setup. We conduct experiments on two testbeds: a 2-node cluster for microbenchmarks and an 8-node cluster for deployment tests. Each node runs Ubuntu 22.04 with Linux kernel 5.15, Kubernetes 1.29 with containerd, Calico 3.27.0, and Cilium 1.18.1. All experiments use CUBIC as the congestion control algorithm.

Baselines. We compare BeeQoS against three baselines: (1) **Cilium** [5], using its built-in TC/eBPF-based bandwidth limiter; (2) **TCHTB** [8], the standard Linux hierarchical token bucket configured with per-class guarantees; and (3) **NoShaper**, i.e., best-effort forwarding without any bandwidth control.

Workloads. We use a mix of synthetic and application workloads. Bursty, latency-sensitive flows are generated with `sockperf` [1], while long-lived bulk transfers use `iperf3/iperf` [2]. End-to-end application QoE is evaluated with `GoDash` [31, 34] in a mixed deployment with concurrent download services.

4.2 Priority Guarantees (Q1)

We evaluate BeeQoS under concurrent multi-priority workloads using three pairs of `iperf3` client-servers sharing a 1 Gbps link. Traffic is generated with `sockperf` to emulate web request workloads (packet size: 1472 bytes) across three classes: high-priority (HP), medium-priority (MP), and low-priority (LP). For configuration,

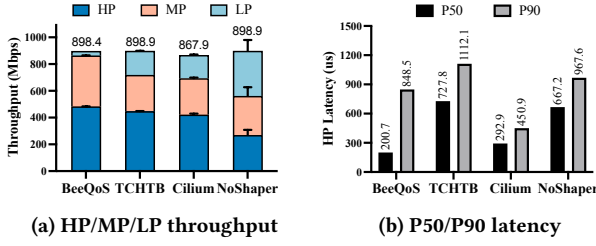


Figure 4: Per-priority throughput (a) and tail latency (b) under concurrent multi-priority traffic.

BeeQoS guarantees 500 Mbps for HP and 300 Mbps for MP flows, while residual bandwidth is distributed according to a priority ratio of 7:5:1 (HP:MP:LP). For Cilium, which lacks native support for priority-based guarantee, we emulate partial guarantees by capping LP flows at 200 Mbps and MP flows at 300 Mbps. However, when HP demand falls below its reserved quota, other classes cannot reclaim the unused bandwidth due to Cilium’s static rate limiting, resulting in significant bandwidth underutilization. Moreover, Cilium does not provide strict priority differentiation among traffic classes. Evaluation metrics include per-priority throughput and tail latency (P50/P90). Each experiment runs for 60 seconds, repeated ten times; we report mean and standard deviation across runs. Figures 4(a) and (b) present the stacked per-priority throughput and tail latency distributions, respectively.

From Figure 4(a), BeeQoS achieves slightly higher HP throughput than TCHTB, substantially higher MP throughput, and lower LP throughput, reflecting its priority-aware allocation. In contrast, both TCHTB and Cilium strictly enforce static rate limits, leading to rigid bandwidth distribution, while NoShaper provides no isolation and allows all flows to compete freely. Notably, Cilium fails to fully utilize the link, resulting in significantly lower total throughput compared to the other schemes. BeeQoS improves HP throughput by 7.9%, 14.6%, and 79.2% over TCHTB, Cilium, and NoShaper, and MP throughput by 35.8%, 36.4%, and 33.2%, respectively. From Figure 4(b), BeeQoS delivers the lowest P50 latency for HP flows and the second-lowest P90 latency, owing to its efficient data plane and adaptive allocation. TCHTB incurs higher latency due to hierarchical token-bucket queuing, while NoShaper suffers from severe latency spikes for HP flows caused by uncontrolled contention. Cilium achieves the lowest P90 latency, primarily because it underutilizes the link and thus faces less pressure. Overall, BeeQoS reduces P50 latency by 72.4%, 31.5%, and 69.9% compared to TCHTB, Cilium, and NoShaper, respectively, and lowers P90 latency by 23.7% and 12.3% compared to TCHTB and NoShaper.

BeeQoS stands out by enforcing strict priority guarantees: delivering high throughput and low tail latency for HP flows while flexibly allocating bandwidth across MP and LP traffic.

4.3 Adaptivity to Demand Shifts (Q2)

We evaluate BeeQoS under dynamic HP demand using an iperf3 client-server setup with two traffic classes (HP and LP flows) sharing a 1 Gbps link. The HP guaranteed rate is set to 600 Mbps. An LP flow starts at 0 s and attempts to fully occupy the link, while HP demand varies across phases: 400 Mbps at 10 s, 800 Mbps at 20 s, 400 Mbps

at 30 s, and terminates at 40 s. Each experiment runs ten times, and we report the mean and standard deviation of throughput. Figures 5(a) and (b) show the time-series throughput of HP and LP flows under dynamic HP demand, and Figure 5(c) summarizes the average bandwidth. For Cilium, since it does not support priority-aware rate control, we emulate a pseudo-guarantee for HP flows by capping LP traffic at 400 Mbps.

From Figure 5(a), BeeQoS and TCHTB both provide strong protection for HP flows, delivering rapid guarantees and fast responsiveness. In contrast, Cilium achieves HP protection only indirectly by capping LP at 400 Mbps, which limits its ability to fully utilize the link. NoShaper fails to isolate HP traffic, leading to bandwidth contention with LP during 20–30 s. Under this high-demand phase, BeeQoS improves HP throughput by 51.8% over Cilium and 70.5% over NoShaper. From Figure 5(b), TCHTB achieves the highest LP throughput since its design allows opportunistic borrowing from the parent’s token bucket. BeeQoS, with its dynamic regulation, induces moderate LP fluctuations and leaves some bandwidth underutilized. NoShaper allows LP traffic to substantially exceed its fair share, while Cilium enforces a strict 400 Mbps cap without prioritization, preventing LP from exploiting unused HP bandwidth. BeeQoS improves LP throughput by 30.9% over Cilium. Figure 5(c) shows average link bandwidth: TCHTB achieves 933 Mbps, BeeQoS reaches 820.1 Mbps, while Cilium lags at 647.4 Mbps. BeeQoS improves average bandwidth utilization by 26.7% over Cilium.

Unlike Cilium and NoShaper, which either waste bandwidth or fail to provide priority isolation, BeeQoS dynamically reallocates resources to guarantee HP performance while regulating LP usage. While TCHTB achieves slightly higher LP throughput, BeeQoS delivers comparable utilization with far lower complexity, making it a more practical choice for dynamic cloud workloads.

4.4 Concurrency Performance (Q3)

We evaluate BeeQoS under high concurrency within a single service class to assess its data plane scalability and efficiency. The experiment runs N concurrent flows ($N = 8, 32, 128, 512, 1024$), where each flow corresponds to a TCP connection. Traffic is generated using iperf at the specified concurrency levels to emulate realistic fan-out scenarios. For each setting, we conduct ten 120-second trials and report the mean and standard deviation of total link throughput, per-flow throughput, and Jain’s fairness index [33]. For BeeQoS and TCHTB, all flows are configured as high priority with a guaranteed bandwidth of 600 Mbps to activate their shaping mechanisms. Since Cilium does not support bandwidth guarantees, we configure its rate limit at the full 1 Gbps link capacity, which in fact gives it a slight advantage. Figure 6 summarizes the results: (a) average total throughput versus the number of concurrent flows, and (b) per-flow Jain’s fairness index (0–1, higher is better).

From Figure 6(a), as concurrency increases, both BeeQoS and NoShaper sustain near-saturation of the link, maintaining stable throughput. In contrast, TCHTB exhibits a steady decline, achieving only 681.2 Mbps at 1024 flows, compared to 925.8 Mbps for BeeQoS. Across high-concurrency, BeeQoS outperforms TCHTB in total throughput by 20–36%. Although Cilium is configured with a rate limit matching the link capacity, its throughput remains lower than expected, likely due to overhead introduced by its complex design.

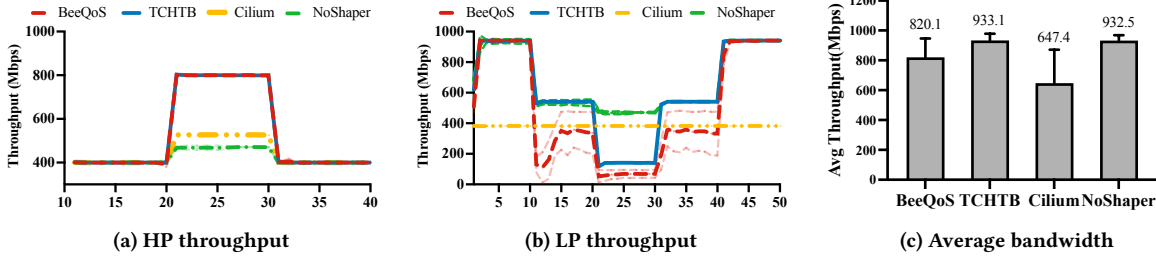


Figure 5: Time-series throughput and average throughput under dynamic HP demand. Light red bands show BeeQoS error bars.

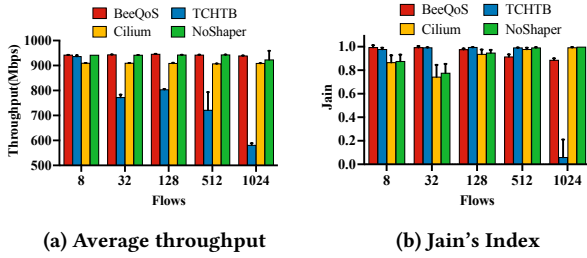


Figure 6: Average throughput and fairness under increasing flow concurrency.

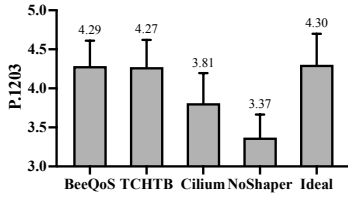


Figure 7: Average DASH video QoE (ITU-T P.1203, 0-5, higher is better) under concurrent web downloads. Video flows are high priority, while download flows are low priority.

From Figure 6(b), BeeQoS maintains consistently stable per-flow fairness, demonstrating the smooth rate-limiting behavior enabled by its high-performance, eBPF-based EDT data path. NoShaper exhibits an initial drop followed by a gradual recovery in fairness under the influence of the congestion control algorithm, eventually stabilizing. Cilium shows a similar fairness trend to NoShaper, likely because its rate limit is set to the link capacity. TCHTB achieves good fairness with a small number of flows, but its fairness deteriorates sharply beyond 1024 flows, coinciding with the low throughput observed earlier and showing its scalability limitations.

BeeQoS sustains high throughput while maintaining stable per-flow fairness under extreme concurrency, showcasing the efficiency, scalability, and responsiveness of its eBPF-based dynamic rate-control data path compared to static or less flexible alternatives.

4.5 Mixed-Application QoE Evaluation (Q4)

We evaluate BeeQoS under a mixed workload where video streaming is high priority and bulk downloads are low priority. The setup includes 100 DASH clients (goDash [31, 34]) and 100 concurrent downloads (iperf3) sharing a 1 Gbps link. As a reference, an **Ideal**

baseline runs only the video flows. Video QoE is measured with ITU-T P.1203 [35] (0-5 scale) on copyright-free 1080p content. For fairness, BeeQoS and TCHTB each reserve 600 Mbps guaranteed bandwidth for video; Since Cilium lacks priority-based shaping, we approximate protection by capping low-priority flows at 400 Mbps. NoShaper is an uncontrolled baseline. We compare how well each scheme preserves video QoE under competing traffic.

Figure 7 shows that BeeQoS and TCHTB provide the strongest protection for video streams, achieving QoE scores close to the **Ideal**, likely due to stable bandwidth allocation. In contrast, Cilium achieves a lower QoE of 3.81, and NoShaper performs worst at 3.37. Compared to Cilium and NoShaper, BeeQoS improves video QoE by 12.6% and 27.3%, respectively. Overall, BeeQoS delivers the highest video QoE, demonstrating its effectiveness in protecting latency-sensitive traffic under competing background flows.

5 Conclusion

We presented BeeQoS, a cloud-native QoS system integrating an eBPF-powered data plane, a demand-aware control plane, and Kubernetes-native deployment. BeeQoS enforces multi-priority guarantees, adapts to dynamic demand, and scales efficiently under high concurrency. From extensive experiments, BeeQoS improves high- and medium-priority throughput, reduces latency, maintains performance even at 1K+ flows, reduces deployment overhead, while also boosting video QoE. These results demonstrate BeeQoS's practicality and superiority over traditional rate-limiters, providing a deployable foundation for adaptive cloud-native QoS.

6 Acknowledgements

We would like to sincerely thank Changye Wu and Weibin Kong for their helpful assistance throughout this work. This work is supported by the National Natural Science Foundation of China (Grant 62422208, 62232010), Ministry of Industry and Information Technology of China (Grant TC240A9ED-70), Shandong Science Fund (Grant ZR2025LZH006), Research Project of Quancheng Laboratory, China (Grant QCL20250106), Project of the Major Innovation Project of Key Laboratory of Computing Power Network and Information Security, Ministry of Education (Grant 2024ZD012); in part by Taishan Scholars Program under Grant (tsqn202507042); in part by National Natural Science Foundation of China (Grant 62472253, U23A20273); in part by Shandong Provincial Natural Science Foundation under Grant ZR2025LZH005; in part by the Taishan Scholars Program under Grant tsqn202408025.

References

- [1] A network benchmarking utility over socket API that was designed for testing performance. <https://github.com/Mellanox/sockperf>.
- [2] A tool for active measurements of the maximum achievable bandwidth on IP networks. <https://iperf.fr/>.
- [3] Amazon-VPC-CNI. <https://aws.amazon.com/cn/blogs/containers/ensuring-fair-bandwidth-allocation-for-amazon-eks-workloads>.
- [4] Calico: Configure QoS Controls. <https://docs.tigera.io/calico/latest/networking/configuring/qos-controls>.
- [5] Cilium: eBPF-based networking, observability, security. <https://cilium.io/>.
- [6] CNI bandwidth plugin. <https://www.cni.dev/plugins/current/meta/bandwidth/>.
- [7] Flannel is a simple and easy way to configure a layer 3 network fabric designed for Kubernetes. <https://github.com/flannel-io/flannel>.
- [8] HTB - Hierarchy Token Bucket. <https://www.man7.org/linux/man-pages/man8/tc-htb.8.html>.
- [9] Katran: A high-performance layer 4 load balancer. <https://github.com/facebookincubator/katran>.
- [10] Kubernetes: Support traffic shaping. <https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/network-plugins/>.
- [11] libbpf. <https://github.com/libbpf/libbpf>.
- [12] Linux Programmer's Manual. tc-bpf(8). <https://man7.org/linux/man-pages/man8/tc-bpf.8.html>.
- [13] Overview of the colocation of different types of workloads. <https://help.aliyun.com/zh/ack/ack-managed-and-ack-dedicated/user-guide/overview-of-the-colocation-of-different-types-of-workloads/>.
- [14] Replacing HTB with EDT and BPF. <https://netdevconf.info/0x14/session.html?talk-replacing-HTB-with-EDT-and-BPF>.
- [15] TC: Traffic Control in the Linux kernel. <https://www.man7.org/linux/man-pages/man8/tc.8.html>.
- [16] TCP: Switch to Early Departure Time model. <https://lwn.net/Articles/766564/>.
- [17] The IO Visor Project. eXpress Data Path (XDP). <https://www.iovisor.org/technology/xdp>.
- [18] The Linux kernel development community. <https://docs.kernel.org/networking/skbuff.html>.
- [19] VPC-CNI for container network. <https://cloud.tencent.com/document/product/457/50353>.
- [20] Soumya Antony, Simy Antony, A. S. Ajeena Beegom, and M. S. Rajasree. HTB based packet scheduler for cloud computing. In *Proc. of ACM ICCSEIT*, 2012.
- [21] Neal Cardwell, Yuchung Cheng, C Stephen Gunn, Soheil Hassas Yeganeh, and Van Jacobson. Bbr: Congestion-based congestion control: Measuring bottleneck bandwidth and round-trip propagation time. *Queue*, 14(5):20–53, 2016.
- [22] Mo Dong, Qingxi Li, Doron Zarchy, P Brighten Godfrey, and Michael Schapira. PCC: Re-architecting congestion control for consistent high performance. In *Proc. of USENIX NSDI*, 2015.
- [23] Songshi Dou, Li Qi, and Zehua Guo. ARES: Predictable traffic engineering under controller failures in SD-WANs. In *Proc. of ACM WWW*, 2024.
- [24] Toke Høiland-Jørgensen, Jesper Dangaard Brouer, Daniel Borkmann, John Fastabend, Tom Herbert, David Ahern, and David Miller. The express data path: Fast programmable packet processing in the operating system kernel. In *Proc. of ACM CoNEXT*, 2018.
- [25] Bert Hubert et al. Linux advanced routing & traffic control howto. *Netherlabs BV*, 1(0):99–107, 2002.
- [26] Chengjun Jia, Zhe Fu, Xiaohe Hu, Shui Cao, Liang Wang, and Jun Li. Multi-core HTB for bandwidth sharing. In *Proc. of ACM/IEEE ANCS*, 2018.
- [27] Alok Kumar, Sushant Jain, Uday Naik, Anand Raghuraman, Nikhil Kasinadhuni, Enrique Cauch Zermeno, C. Stephen Gunn, Jing Ai, Björn Carlin, Mihai Amarandei-Stavila, Mathieu Robin, Aspi Siganporia, Stephen Stuart, and Amin Vahdat. BwE: Flexible, hierarchical bandwidth allocation for WAN distributed computing. In *Proc. of ACM SIGCOMM*, 2015.
- [28] Chang Liu, Byungchul Tak, and Long Wang. Understanding performance of eBPF maps. In *Proc. of ACM SIGCOMM WKSHPs*, 2024.
- [29] Steven McCanne and Van Jacobson. The BSD Packet Filter: A new architecture for user-level packet capture. In *Proc. of USENIX winter*, 1993.
- [30] Sebastiano Miano, Xiaoqi Chen, Ran Ben Basat, and Gianni Antichi. Fast in-kernel traffic sketching in eBPF. *ACM SIGCOMM Computer Communication Review*, 53(1):3–13, 2023.
- [31] John O'Sullivan, Darjo Raca, and Jason J Quinlan. godash 2.0-the next evolution of has evaluation. In *Proc. of IEEE WoWMoM*, 2020.
- [32] Shixiong Qi, Leslie Monis, Ziteng Zeng, Ian-chin Wang, and KK Ramakrishnan. Spright: Extracting the server from serverless computing! High-performance eBPF-based event-driven, shared-memory processing. In *Proc. of ACM SIGCOMM*, 2022.
- [33] D. Chiu R. Jain and W. Hawe. A quantitative measure of fairness and discrimination for resource allocation in shared systems. *DEC Research, Technical Report*, 1984.
- [34] Darjo Raca, Maëlle Manificier, and Jason J Quinlan. Godash-go accelerated has framework for rapid prototyping. In *Proc. of IEEE QoMEX*, 2020.
- [35] Werner Robitz, Steve Göring, Alexander Raake, David Lindegren, Gunnar Heikkilä, Jörgen Gustafsson, Peter List, Bernhard Feiten, Ulf Wüstenhagen, Marie-Neige Garcia, et al. HTTP adaptive streaming QoE estimation with ITU-T rec. P. 1203: Open databases and software. In *Proc. of ACM MMSys*, 2018.
- [36] Dong-Hyuk Seo, Hyomin Shin, and Sang-Wook Kim. Exploring search volumes of terms in web portals for accurate event-aware traffic prediction. In *Proc. of ACM WWW*, 2025.
- [37] Zhenning Shi, Dan Zhao, Yijia Zhu, Guorui Xie, Qing Li, and Yong Jiang. Helios: Learning and adaptation of matching rules for continual in-network malicious traffic detection. In *Proc. of ACM WWW*, 2025.
- [38] Rongrong Wang, Duc Van Le, Jikun Kang, Rui Tan, and Xue Liu. Incentive temperature control for green colocation data centers via reinforcement learning. In *Proc. of IEEE/ACM IWQoS*, 2024.
- [39] Yu Zhang, Qingsong Wei, Cheng Chen, Mingdi Xue, Xinkun Yuan, and Chundong Wang. Dynamic scheduling with service curve for QoS guarantee of large-scale cloud storage. *IEEE Trans. on Computers*, 67(4):457–468, 2017.

A Parameter Configuration

BeeQoS exposes tunable parameters that govern controller pace, sensitivity, and allocation. In our implementation, the *sampling interval* is 10 ms and the *control interval* $10\times$ larger, providing frequent per-flow sampling and stable rate control; for wider-area deployments, it should scale with the RTT. Key parameters include *Throttle threshold* $\theta = 0.9$, *Action factor* $\beta = 0.1$, and *Selection factor* $\gamma = 0.7$, which influence convergence and allocation: larger β accelerates bandwidth acquisition, smaller θ increases responsiveness. These parameters are robust within reasonable ranges; in practice, $\beta \in [0.05, 0.2]$ and moderate control-interval variations maintain stable, predictable behavior.

B Scalability and Deployment Strategy

BeeQoS is deployed as a privileged DaemonSet on each Kubernetes node. Each daemon continuously runs the dynamic allocation algorithm, adaptively distributing bandwidth among local workloads. Policies can be dynamically updated via the CLI and automatically applied across rescheduled containers, enabling seamless and large-scale deployment.

C Ease of Deployment (Q5)

We evaluate the deployability of BeeQoS against TCHTB on Kubernetes clusters of 2 and 8 nodes. Configuration effort is measured as the minimal number of required commands or units: each Pod annotation counts as one unit, while each TC class, filter, or command counts as one command. Cloud-native commands unrelated to QoS are excluded. Table 1 summarizes the results.

Table 1: Minimal configuration commands or units for initial deployment and Pod migration in 2-node and 8-node clusters.

Scheme	Initial Deployment		Pod Migration
	2-node	8-node	node1→node2
BeeQoS (annotations)	2	2	0
TCHTB (TC commands)	20	80	8

For initial deployment, BeeQoS requires only two Pod annotations (ingress and egress), applied cluster-wide with a single `kubect1` apply. The cost remains constant regardless of cluster size. In contrast, TCHTB requires 10 manual commands per node (adding qdiscs, classes, filters, and IFB setup), yielding 20 commands

on 2 nodes and 80 on 8 nodes. Pod migration further widens the gap. BeeQoS needs only one `kubectl apply`, without touching bandwidth annotations. TCHTB requires at least eight commands per Pod: removing ingress/egress filters on the source node and re-adding root/subclass/filter rules on the destination node (assuming IFB devices persist). All steps require `sudo`.

BeeQoS eliminates per-node and per-Pod configuration, achieving constant deployment cost and zero-touch migration. In contrast, TCHTB's overhead grows linearly with cluster size and Pod movement, showing the operational burden of traditional rate limiting.